



Semester Schedule Generator

Amir salah and Omar Alnaimat

Supervised By:
Dr. Anas AL-Bdareen

The project is presented for the degree of Bachelor in Computer Science
and Artificial Intelligence and Software Engineering

Faculty of Information Technology
Aqaba University of Technology
Winter 2025-2026

Declaration of Originality

This document was written entirely by the undersigned team members of the project. The source of every quoted text is clearly cited, and there is no ambiguity in where the quoted text begins and ends. The source of any illustration, image, or table that is not the work of the team members is also clearly cited. We are aware that using non-original text or material, or paraphrasing or modifying it without proper citation, is a violation of university regulations and is subject to legal action.

Furthermore, we confirm that all the analysis, design decisions, implementation details, and architectural components presented in this project were produced by the team members based on our own understanding and development process. No external code or content was used without proper acknowledgment, and every referenced work is appropriately listed in the references section. By submitting this document, we declare that the work reflects our personal effort and complies fully with the academic integrity standards of the university.

Acknowledgments

We would like to express our sincere gratitude to everyone who supported the development of the Semester Schedule Generator project. Our deepest appreciation goes to our supervisor, Dr. Anas AL-Bdareen, for providing continuous guidance, valuable feedback, and essential insights that helped shape the direction of this work.

We also extend our thanks to the academic staff who offered clarification on scheduling policies and institutional requirements, which greatly assisted us in designing realistic system rules and workflow constraints. Special appreciation is given to the university personnel who facilitated access to academic data structures and supported our understanding of registration processes.

Finally, we are grateful to our families, friends, and colleagues for their encouragement and motivation throughout the duration of this project. Their support played a crucial role in helping us complete this work successfully.

– Project Members
Amir Salah
Omar Alnaimat

Contents

1	Introduction	2
1.1	Overview	2
1.2	Background and Related Work	2
1.2.1	Background	2
1.2.2	Related Work	3
1.3	Problem Statement	4
1.4	Aim and Objectives	4
1.4.1	Project Objectives	5
1.4.2	System Objectives	5
1.5	Project Scope and Limitations	6
1.6	Gantt Chart	7
2	Project Plan	10
2.1	Project Deliverables	10
2.2	Project Tasks	12
2.3	Risk Assessment	13
2.4	Project Management Tools	14
3	Requirements Specification	17
3.1	Overview	17
3.2	Scope	17
3.3	Actor	17
3.4	Overall Description	17
3.4.1	System Overview	17
3.4.2	Operating Environment	18
3.5	Functional Requirements	18
3.6	Use Case Specification	19
3.6.1	Use Case: Login / Register	19
3.6.2	Use Case: Schedule Generation	19
3.6.3	Use Case: View Generated Schedule	20
3.6.4	Use Case: Logout	20
3.7	Non-Functional Requirements	20
3.7.1	Performance	20
3.7.2	Reliability	20
3.7.3	Usability	21
3.7.4	Security	21
3.7.5	Maintainability	21
3.8	Constraints	21
3.9	Platform Requirements	21
3.9.1	Client-Side Requirements	21
3.9.2	Software Requirements	22
3.9.3	Functional Capabilities	22
3.9.4	Performance Requirements	22
3.9.5	Server-Side Requirements	22
3.9.6	Hardware Requirements	22
3.9.7	Network Requirements	24

3.9.8	Integration Requirements	24
4	System Design	26
4.1	Overview	26
4.2	Logical Design	26
4.2.1	Sequence diagram	26
4.2.2	System Data Flow Overview	30
4.2.3	ER Diagram	32
4.2.4	Schema	33
5	Implementation	35
5.1	Overview	35
5.2	Implementation Environment	35
5.3	Project File Structure	36
5.4	System Configuration	37
5.5	Data Acquisition and ETL Module	38
5.5.1	API Data Fetching	38
5.5.2	Data Transformation	38
5.5.3	Database Loading with MERGE	39
5.6	Scheduling Engine	40
5.6.1	Student Analysis and Graduation Detection	40
5.6.2	Course Demand Calculation	41
5.6.3	Course Opening Decision	41
5.6.4	Instructor Assignment	42
5.6.5	Instructor Load Rules	42
5.6.6	Room and Time Slot Assignment	43
5.7	Web User Interface	43
5.8	PDF Report Generation	47
5.9	System Workflow During Execution	47
5.10	Implementation Challenges	48
5.11	Summary	48
6	Testing and Validation	50
6.1	Overview	50
6.2	Testing Objectives	50
6.3	Testing Environment	50
6.4	Unit Testing	51
6.4.1	Graduation Detection Test	51
6.4.2	Prerequisite Validation Test	51
6.4.3	Instructor Load Test	51
6.4.4	Force-Open Logic Test	52
6.5	Integration Testing	52
6.6	Functional Testing	53
6.7	Constraint Validation Testing	54
6.8	Performance Testing	54
6.9	Bug Fixing During Testing	55
6.10	Bug Fixing During Testing	56
6.11	Testing Result Summary	56
6.12	Summary	56

7	Conclusion and Future Work	58
7.1	Conclusion	58
7.2	Project Achievements	58
7.3	Limitations Revisited	59
7.4	Future Work	60
7.5	Final Remarks	61
	References	63

List of Figures

1	Use Case Diagram for Schedule Generator System	18
2	Login	26
3	Schedule generation	27
4	View schedule	28
5	Logout	29
6	Data Flow Between System Components Level 0	30
7	Data Flow Between System Components Level 1	31
8	Entity–Relationship Diagram of the Semester Schedule Generator System	32
9	Database Schema for the Semester Schedule Generator System	33
10	Login Page — Administrator Authentication	44
11	Dashboard Page — Live System Statistics	44
12	Semester Schedule Table with Inline Editing	45
13	Semester Summary Report — Course Demand and Section Coverage .	45
14	Analytics Page — Student Demand, Room Utilization, and Day Pattern Distribution	46
15	Run Pipeline Page — Executes the Full Scheduling Pipeline	46

List of Tables

1	Gantt Chart (Weeks 1–16)	7
2	Gantt Chart (Weeks 17–20)	8
3	Gantt Chart (Weeks 20–24)	8
4	Instructor Teaching Load Limits	42
5	Testing Environment Specifications	50
6	Functional Test Cases	53
7	Constraint Validation Results	54
8	Performance Test Results	54
9	Bugs Found and Fixed During Testing	55

Abstract

The Semester Schedule Generator project presents an automated and intelligent solution for producing accurate, conflict-free academic schedules for university students. Traditional manual scheduling processes often lead to errors, overlapping courses assignments, unmet prerequisite requirements, and delays during registration—especially as student numbers and course offerings grow each semester. The proposed system addresses these issues by generating optimized schedules based on academic rules, student progress, prerequisite validation, credit limits, and the priority of graduating students.

In this system, student data is obtained through a custom adapter that retrieves the necessary information from the university’s database and stores it locally for processing. This approach ensures data accuracy while allowing the scheduling engine to work independently from the institution’s internal systems. Instead of building full student profiles in advance, the system adopts an On-the-Fly and lazy evaluation mechanism that constructs schedules only when required. The algorithm automatically identifies courses that each student must still complete and matches them with the officially offered courses for the semester.

Overall, the Semester Schedule Generator provides a flexible, efficient, and scalable solution for academic scheduling. It minimizes manual effort, reduces data redundancy, and supports fast, accurate decision-making—making it a practical tool for modern educational environments.

Keywords— On-the-Fly Generation, Lazy Evaluation, Database Integration, Adapter-Based Data Retrieval.

]

CHAPTER 01
INTRODUCTION

1 Introduction

1.1 Overview

This chapter provides an introduction to the Semester Schedule Generator, explaining the challenges of traditional manual scheduling and the motivation behind developing an automated solution. It outlines the background of academic scheduling, defines the problem the system aims to solve, and presents the overall aim and objectives of both the project and the system. The chapter also clarifies the scope and limitations of the work, establishing the foundation for the detailed planning, requirements, and system design discussed in later chapters.

1.2 Background and Related Work

1.2.1 Background

In many academic institutions, the process of creating semester schedules is still performed manually or with limited automation, requiring significant time and effort from academic staff. This process involves examining each student's educational record, determining eligible courses, verifying prerequisite completion, and ensuring that no time conflicts occur. As universities expand and academic structures become more complex, manual scheduling becomes increasingly difficult to manage effectively.

Students follow different academic paths and complete courses at varying rates, making it challenging to create individualized schedules that accurately reflect each student's progress. Manual scheduling also increases the risk of errors, such as assigning courses without meeting prerequisites, creating overlapping class times.

These recurring issues reveal the need for solutions that can process large amounts of student data, apply academic rules consistently, and produce accurate schedules with minimal human intervention. Automation improves efficiency, reduces human error, and ensures fairness across the scheduling process.

Our proposed project, the *Semester Schedule Generator*, is designed to address these challenges by providing a rule-based, automated system capable of analyzing student records, applying academic constraints, and generating personalized and conflict-free semester schedules. The system enhances the registration experience by improving accuracy, reducing workload, and supporting reliable academic planning.

1.2.2 Related Work

Smart Timetable Generator [1] presents an automated class scheduling system aimed at replacing manual timetable creation in academic institutions. The project integrates course data, lecturer assignments, room availability, and time-slot constraints into a unified workflow. It focuses on generating conflict-free timetables using rule-based automation. The system reduces human effort and scheduling errors through structured data processing. Its design emphasizes modularity and usability.

Automatic Timetable Generation Using Genetic Algorithm [2] proposes a heuristic-based scheduling approach using Genetic Algorithms to solve academic timetabling problems. The system models timetabling as an NP-complete optimization problem. It automatically assigns courses, rooms, and time slots while minimizing conflicts. The project demonstrates how evolutionary techniques can improve timetable quality. A graphical interface supports administrative interaction.

Implementation of a Time Table Generator Using Visual Basic .NET [3] presents a desktop-based timetable generation system developed for a single university department. The system manages students, lecturers, courses, venues, and periods using an object-oriented design. A randomization-based algorithm is used to generate clash-free lecture and exam timetables. Results can be exported in HTML and Excel formats. The solution targets departmental-level automation.

A Multi-Agent System for Course Timetable Generation [4] introduces a decision-support system based on a Multi-Agent System (MAS) architecture. The project uses collaborative agents to model real-world timetable planning behavior. Agents negotiate instructor preferences, student suggestions, and resource constraints. The system handles both hard and soft constraints. It aims to bridge the gap between theoretical models and real institutional needs.

Time Table Generation System by Biemba [5] proposes a rule-based academic scheduling system using formal software engineering practices. The project evaluates multiple timetabling algorithms, including graph coloring and constraint programming. Constraints are represented using an XML-based structure. The system focuses on maintainability and extensibility. It reduces reliance on manual timetable preparation.

Generating Timetable and Students' Schedule Based on Data Mining Techniques [6] presents a data-driven approach using FP-Tree clustering to generate academic timetables. Student transcript data is analyzed to identify frequently grouped courses. These clusters are then scheduled to minimize conflicts. The system reduces manual transcript analysis. It emphasizes scalability for large student datasets.

Web Application for Automatic Time Table Generation [7] introduces a web-based system for automated timetable creation. Administrators input teacher availability, course details, and workload constraints through an online interface. The system generates weekly schedules while minimizing conflicts. It supports basic institutional requirements. The project focuses on accessibility and ease of use.

Automated Time Table Generation Using Multiple Context Reasoning for University Modules [8] proposes an intelligent scheduling framework based on context-aware reasoning. Each academic constraint is treated as a separate reasoning context. The system dynamically resolves conflicts by switching between contexts. This approach increases flexibility compared to rigid rule-based methods. The study highlights adaptability in complex scheduling environments.

1.3 Problem Statement

The problem addressed in this project is the inefficiency and challenges associated with manually assigning semester courses to university students. The current manual scheduling process is time-consuming, prone to human error, and often results in inconsistencies, unmet prerequisite requirements, misplaced course priorities, and delays for graduating students. As student numbers grow and academic structures become more complex, administrators struggle to manage course allocation accurately and efficiently. This leads to registration conflicts, dissatisfaction among students, and difficulty ensuring smooth academic progression.

Specifically, the project aims to solve the following problems:

- Students being assigned to courses without meeting prerequisite requirements.
- Overlapping course schedules that affect academic progression.
- Difficulty identifying the remaining courses required for each student.
- Lack of prioritization for graduating students who need specific courses to complete their degree.
- High manual workload on administrators due to repetitive scheduling tasks.
- Human errors leading to inconsistent or incorrect course assignments.
- No automated generation of structured schedule reports (e.g., PDF output).
- Redundant or inefficient handling of student data across multiple processes.

1.4 Aim and Objectives

The Semester Schedule Generator aims to automate the assignment of semester courses to university students using academic rules, prerequisite validation, and priority handling, while minimizing manual work and improving accuracy.

1.4.1 Project Objectives

The following project objectives outline the key development, planning, and documentation activities required to complete the Semester Schedule Generator project:

1. To analyze existing scheduling challenges and translate them into clear functional and non-functional requirements.
2. To plan, design, and implement a complete automated semester scheduling solution that reduces manual workload for academic staff.
3. To design the database schema, system workflow, and architecture needed to support automated schedule generation.
4. To document the full development process, including requirements, design, implementation, testing, and evaluation.
5. To produce a final report and presentation that demonstrates the system's functionality and development methodology.

1.4.2 System Objectives

The system objectives define the core functional goals that the Semester Schedule Generator must achieve to ensure accurate, efficient, and reliable automated schedule production. These objectives include:

1. To automatically generate individualized semester schedules without manual intervention.
2. To implement a rule-based scheduling mechanism that validates prerequisites, credit limits, academic progression, and institutional policies.
3. To incorporate a priority-based assignment process that ensures graduating and special-case students receive enrollment precedence.
4. To reduce data redundancy using Dynamic Fetch and On-the-fly Generation instead of storing full student profiles.
5. To generate structured PDF reports automatically after schedule creation, including all relevant course assignment details.
6. To enhance institutional efficiency by minimizing human errors, reducing processing time, and streamlining academic workflow.

1.5 Project Scope and Limitations

Scope:

The scope of the *Semester Schedule Generator* is limited to supporting the academic scheduling requirements of Aqaba University of Technology (AUT). The system is designed to generate automated semester schedules for AUT students by analyzing their academic records, completed and pending courses, prerequisites, and credit constraints.

The system imports student and course data from AUT through a custom-built data adapter and stores the information locally for processing. Based on AUT's academic rules and registration policies, the scheduling engine generates personalized, conflict-free schedules that align with the university's curriculum structure.

The project focuses solely on course assignment and schedule generation. It does not include features such as exam scheduling, attendance tracking, advising systems, or integration with external platforms. All functionalities are tailored specifically to AUT's academic environment and may require adjustments for use in other institutions.

Limitations:

The *Semester Schedule Generator* has several limitations resulting from its scope and design constraints:

1. **No Exam Scheduling**

The system is limited to generating semester course schedules and does not support the creation of midterm or final exam timetables.

2. **No Real-Time Data Integration**

The system relies on imported data and does not provide direct live synchronization with university databases or online systems.

3. **Manual Updates for Academic Changes**

Any modifications to academic rules, prerequisites, or course structures must be performed manually by the system administrator.

4. **Limited Handling of Sudden Adjustments**

Unexpected changes, such as instructor absence or room unavailability, require manual intervention because the system does not support automatic rescheduling.

5. **Static PDF Output**

The PDF schedule generated by the system is static and cannot be edited through the interface. Any modification requires regenerating the schedule.

6. **Not Universally Adaptable**

The system is tailored to a specific academic structure and may require customization before it can be applied in different institutions.

1.6 Gantt Chart

1. Weeks 1–4: Chapter 1 – Introduction

Covers the project background, problem statement, aim, objectives, scope, and limitations.

2. Weeks 4–8: Chapter 2 – Project Plan

Includes schedule planning, work distribution, workflow organization, and project management structure.

3. Weeks 8–12: Chapter 3 – Requirements Specification

Defines functional and non-functional requirements, scheduling rules, system constraints, and use case descriptions.

4. Weeks 12–16: Chapter 4 – System Design

Includes system architecture, workflow diagrams, database schema, and data flow modelling.

5. Weeks 17–20: Chapter 5 – Implementation

Includes development of the scheduling engine, database integration, ETL adapter implementation, API communication, and a PDF/Excel report generator.

6. Weeks 20–23: Chapter 6 – Testing & Validation

Covers system testing, conflict detection evaluation, performance stress testing, and validation of timetable correctness.

7. Weeks 23–24: Chapter 7 – Conclusion and Future Work

Summarizes project outcomes, challenges encountered, achieved goals, and suggestions for future enhancements to improve scalability and automation.

Table 1: Gantt Chart (Weeks 1–16)

Main Task	Sub-Task	Assigned To	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16
Requirement Analysis	Collect scheduling rules	Omar (SE)	X	X	X	X												
	Analyze student data logic	Amir (AI)	X	X	X	X												
Requirements Specification	Functional requirements	Omar (SE)								X	X	X	X	X				
	System behavior (AI logic)	Amir (AI)									X	X	X	X				
Database Design	ERD creation	Omar (SE)												X	X	X	X	X
	Data dictionary	Omar (SE)													X	X	X	X
System Design	Workflow diagrams	Omar (SE)												X	X	X	X	X

Table 2: Gantt Chart (Weeks 17–20)

Main Task	Sub-Task	Assigned To	W17	W18	W19	W20
Implementation (Chapter 5)	API + ETL adapter coding	Amir (AI)	X	X		
	Scheduling engine integration	Amir (AI)	X	X	X	
	Frontend / client connection	Omar (SE)		X	X	X
	PDF / Excel report generator	Amir (AI)			X	X

Table 3: Gantt Chart (Weeks 20–24)

Main Task	Sub-Task	Assigned To	W20	W21	W22	W23	W24
Testing (Chapter 6)	Functional test cases	Omar (SE)	X	X			
	Performance and stress testing	Amir (AI)	X	X	X		
	Conflict validation	Both		X	X		
	Bug fixing and re-testing	Omar (SE)			X	X	
Conclusion & Future Work (Chapter 7)	Final report completion	Amir (AI)				X	X
	Future work documentation	Omar (SE)				X	X
	Final presentation preparation	Both				X	X

CHAPTER 02

PROJECT PLAN

2 Project Plan

The Semester Schedule Generator project follows a structured development plan to ensure that all tasks are completed efficiently and in the correct sequence. The plan is based on a simplified Waterfall approach, where each phase builds on the previous one.

The project begins with gathering requirements and understanding the university's academic rules, prerequisites, and scheduling constraints. After that, the system architecture, database design, workflow diagrams, and data flow models are created to define how the system will operate.

Implementation focuses on developing the data adapter to import university data, building the scheduling engine to generate conflict-free schedules, and creating the PDF report generator. These components are then integrated to ensure they work together smoothly.

Testing is performed to verify the accuracy of generated schedules and ensure all constraints are respected. The project concludes with final documentation and a presentation summarizing the design, implementation, and results.

2.1 Project Deliverables

The *Semester Schedule Generator* project will produce a set of technical and documentation deliverables that represent the full development lifecycle of the system. These deliverables ensure that the project satisfies both academic requirements and functional objectives.

1. Project Documentation

A complete technical report written using Overleaf, containing:

- Introduction and background
- Problem statement
- Aim and objectives
- Scope and limitations
- Related work
- Requirements specification
- System design and architecture
- Database schema and ERD
- Implementation details
- Testing and validation
- Results, discussion, and conclusion

2. System Requirements Specification (SRS)

A structured document that describes:

- Functional and non-functional requirements
- System constraints and scheduling rules
- Use case diagrams and descriptions

3. Data Adapter Module

A custom-built adapter responsible for importing student and course data from the university database. This module validates, cleans, and stores the data into the system's local database for processing.

4. Database Design Package

This includes:

- Entity–Relationship Diagram (ERD)
- Logical and physical database schemas
- Data dictionary describing table fields and relationships

5. Scheduling Engine

The core module responsible for:

- Identifying incomplete or required courses
- Checking prerequisite satisfaction
- Applying academic rules and credit limits
- Prioritizing graduating students
- Generating conflict-free semester schedules

6. PDF Report Generator

A module that automatically generates a structured PDF document containing:

- Student information
- Assigned courses for the semester
- Weekly schedule layout
- Summary of prerequisites and academic progress

7. System Integration Package

Integration of all system components, including:

- Data adapter
- Local database
- Scheduling engine
- PDF generator

Ensuring that they operate together in a unified and automated workflow.

8. System Verification and Testing

A complete testing package including:

- Test plan and test cases
- Unit and integration testing results
- Schedule conflict detection analysis
- Error logs and fixes

9. Final Presentation

A professional presentation summarizing:

- Project motivation and objectives
- Architecture and design
- System workflow and demonstration
- Results and evaluation
- Limitations and future improvements

2.2 Project Tasks

The development of the *Semester Schedule Generator* is organized into a set of structured project tasks. Each task contributes to moving the system from initial analysis to final delivery in a systematic manner.

1. Requirement Gathering and Analysis

Identify scheduling challenges, collect academic rules and university policies, understand course and prerequisite structures, and analyze the format of student and course data received from the university. Define key constraints such as credit limits, section capacities, and graduation priorities.

2. System Requirements Specification

Document functional and non-functional requirements, define system use cases, and specify academic and technical constraints that the system must satisfy. This includes requirements related to schedule generation, data importing, validation, and reporting.

3. Data Acquisition and Adapter Development

Develop a data adapter responsible for importing student and course information from the university's database into the local system. This includes validating, cleaning, and transforming the data into a suitable internal format before storage.

4. Database Design

Design the Entity–Relationship Diagram (ERD), define logical and physical database schemas, and create the tables required for students, courses, prerequisites, schedules, and logging. Prepare a data dictionary to describe the structure and meaning of each field.

5. System Design

Design the overall architecture of the Semester Schedule Generator, including the interaction between the data adapter, scheduling engine, database layer, and PDF report generator. Create data flow diagrams and workflow models to describe how data moves through the system.

6. Implementation

- **Scheduling Engine Development:** Implement the core logic that identifies required and pending courses, checks prerequisite satisfaction, applies academic rules and graduation priorities, and generates conflict-free schedules.
- **PDF Report Generator:** Implement a module that produces structured PDF reports summarizing each student's assigned courses and weekly schedule layout.

7. System Integration

Integrate the data adapter, database, scheduling engine, and PDF generator into a unified workflow. Ensure that data flows correctly between components from import to schedule generation and final report creation.

8. Testing and Validation

Design and execute test cases for individual modules and for the full system. Validate that generated schedules are conflict-free, respect prerequisites and credit limits, and handle edge cases such as graduating students and high-demand courses.

9. Documentation

Prepare the complete project documentation using Overleaf, including system requirements, architecture, database design, algorithms, implementation details, and testing results. Include diagrams, tables, and figures where appropriate.

10. Final Presentation and Delivery

Develop a presentation that summarizes the motivation, design, implementation, and results of the Semester Schedule Generator. Demonstrate the system workflow and discuss limitations and potential future enhancements.

2.3 Risk Assessment

The following risks may affect the development and performance of the Semester Schedule Generator system:

- **Data Inconsistency Risk**

Student or course data received from the university database may contain missing or outdated information.

Likelihood: Medium — Impact: High

Mitigation: Apply strict validation rules, log incomplete records, and ensure periodic synchronization.

- **Scheduling Conflicts**

Complex scenarios may result in overlapping course assignments or invalid schedules.

Likelihood: Low-Medium — Impact: High

Mitigation: Add conflict detection, validate constraints, and test using real-world datasets.

- **Performance Issues**

Large student volumes may slow down the scheduling process.

Likelihood: Medium — Impact: Medium

Mitigation: Optimize processing logic, use on-demand generation, and cache metadata.

- **Dependency on External Systems**

The system relies on the university database and API availability.

Likelihood: High — Impact: Medium

Mitigation: Maintain temporary data snapshots, use fallback modes, and schedule stable retrieval times.

- **Incorrect Prerequisite Handling**

Misinterpreted academic rules may cause invalid course assignments.

Likelihood: Low — Impact: High

Mitigation: Validate prerequisites in multiple stages and test rule logic thoroughly.

- **PDF Generation Errors**

Report generation may fail due to formatting or missing data.

Likelihood: Low — Impact: Medium

Mitigation: Validate data before export and use consistent PDF templates.

- **Project Timeline Delays**

Development or documentation progress may fall behind schedule.

Likelihood: Medium — Impact: Medium

Mitigation: Track progress weekly and distribute workload effectively.

2.4 Project Management Tools

Effective project management requires the use of appropriate tools to organize tasks, track progress, collaborate efficiently, and manage project documentation. The Semester Schedule Generator project utilizes a set of tools throughout its development lifecycle.

1. **Microsoft Project**

Used for project scheduling, task breakdown, and timeline management. This tool helps structure the project into well-defined phases, assign responsibilities, set task dependencies, and monitor progress using Gantt charts and critical path analysis.

2. **Overleaf**

A LaTeX-based documentation platform used for writing and formatting the project report. It provides real-time collaboration and structured academic formatting.

3. **Draw.io / Lucidchart**

Utilized for designing diagrams such as ERD, system architecture, data flow diagrams, and workflow models to represent the system visually.

4. **GitHub / GitLab**

Version control tools are used to manage source code, maintain a version history, facilitate collaboration on development, and ensure code safety.

5. **VS Code (Visual Studio Code)**

The primary development environment for implementing the system. VS Code is used for writing the scheduling engine, API integration, adapter module, and database-related scripts. It also supports extensions for Python, SQL, Git, and debugging tools, making it suitable for the entire development workflow.

6. **Database Tools (SQL Server / SQL Viewer Extensions)**

A database management tool used to analyze university database structures, run SQL queries, test data retrieval logic, and validate database connections.

CHAPTER 03

Requirements Specification

3 Requirements Specification

3.1 Overview

The purpose of this system is to automate and optimize the process of generating Semester course offerings in universities. The system aims to replace the manual, time-consuming effort performed by the Registration Department and College Heads by Using structured data, constraints, and role-based algorithms to produce an optimal, Conflict-free semester plan.

3.2 Scope

The system analyzes student data, graduation requirements, instructor teaching loads, classroom availability, and time constraints to generate automatically:

- List of courses to be offered next semester.
- Optimal scheduling of lecture times.
- Automatic assignment of classrooms and instructors.
- Balanced teaching load distribution based on academic rank and department roles.
- Identification of graduating students based on completed credit hours.

3.3 Actor

Administrator(head register)

3.4 Overall Description

3.4.1 System Overview

The System Overview consists of:

- A local database hosted on Visual Studio (SQL Server).
- A Python-based algorithm module integrated into the same application.
- Scheduling and optimization engine.
- Admin dashboard to input constraints and generate the semester plan.
- Exportable timetable and course offering outputs.

3.4.2 Operating Environment

- Backend: SQL Server
- AI Module: Python (integrated into the application)
- Frontend: Frontend: Web Interface
- Operating System: Windows

3.5 Functional Requirements

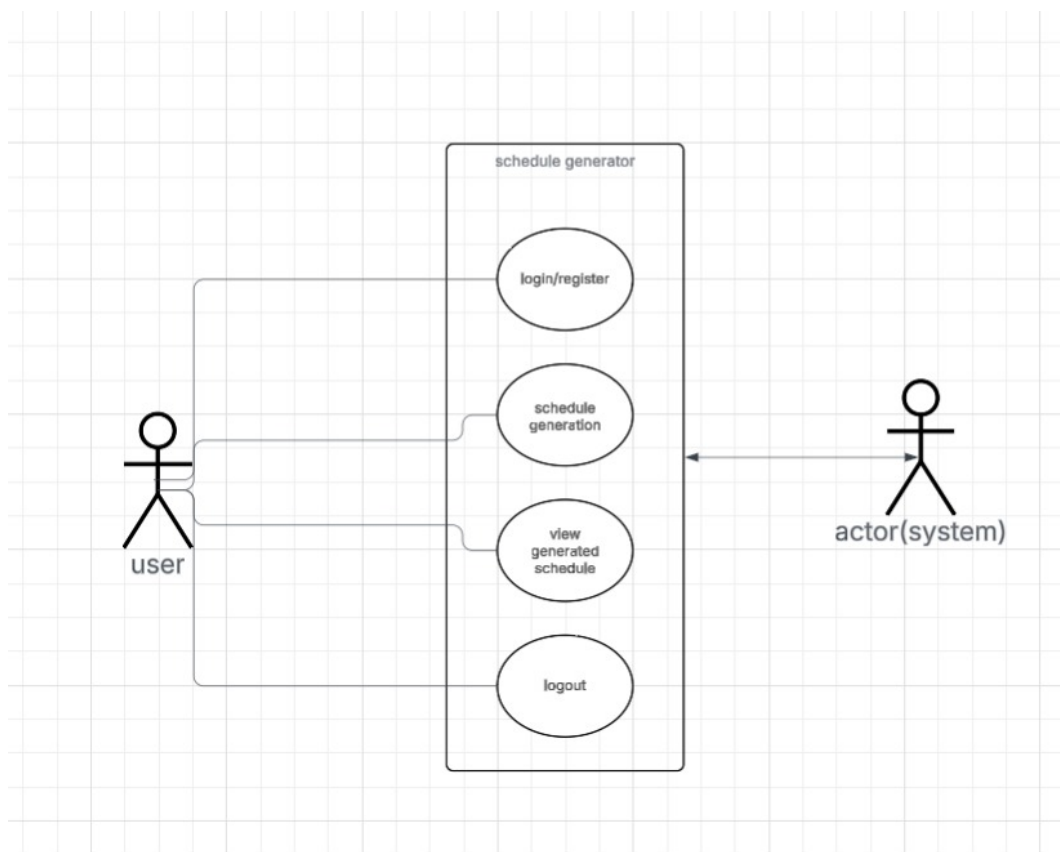


Figure 1: Use Case Diagram for Schedule Generator System

Description:

This diagram represents a use case model for a Schedule Generator system. The user interacts with the system to register or log in, generate a schedule, view the generated schedule, and log out. All functionalities are contained within the Schedule Generator system boundary, showing how the user accesses each feature.

3.6 Use Case Specification

3.6.1 Use Case: Login / Register

Actor: Admin(Registration department head)

Precondition: User must have system access credentials. System is running.

Basic Flow:

1. User selects Login/Register.
2. System validates user credentials or creates a new account.
3. User successfully logs in to the system.
4. User proceeds to generate or view schedules.

Alternate Flows:

- **A1: Invalid Login Credentials** — System displays an error and requests re-entry.
- **A2: Registration Failure** — Missing/invalid data prompts correction request.
- **A3: Session Timeout** — System logs out user automatically due to inactivity.

3.6.2 Use Case: Schedule Generation

Actor: Admin(Registration department head)

Precondition: User logged in and data loaded in the database.

Basic Flow:

1. User requests schedule generation.
2. System fetches student records, courses, instructors, and rooms.
3. AI scheduling module processes student needs and constraints.
4. System allocates rooms, timeslots, and instructors.
5. Final schedule is generated and stored.
6. The system notified the user that the schedule was generated

3.6.3 Use Case: View Generated Schedule

Actor: Admin(Registration department head)

Precondition: A schedule must exist in the system.

Basic Flow:

1. User selects "View Schedule".
2. System retrieves the generated schedule.
3. User views schedule by department/instructor/classroom.

3.6.4 Use Case: Logout

Actor: Admin(Registration department head)

Precondition: User is authenticated in the system.

Basic Flow:

1. User clicks logout.
2. System ends session and redirects to login.

3.7 Non-Functional Requirements

3.7.1 Performance

NFR-1: The system shall generate a complete semester schedule in less than 120 minutes.

NFR-2: The AI scheduling engine shall be capable of processing up to 10,000 student records without performance degradation.

3.7.2 Reliability

NFR-3: The system shall maintain data integrity during all data processing and scheduling operations.

NFR-4: The system shall provide backup and restore mechanisms to prevent data loss.

3.7.3 Usability

NFR-5: The system interface shall be simple and intuitive for non-technical users, such as registration and academic staff.

NFR-6: All generated outputs shall be clearly formatted to support easy review and verification.

3.7.4 Security

NFR-7: Access to scheduling and administrative features shall be restricted to authorized users only.

NFR-8: Student academic records shall be encrypted while stored in the database.

3.7.5 Maintainability

NFR-9: The system shall support the addition of new academic rules and constraints with minimal modification.

NFR-10: The AI scheduling module shall be configurable and modular, avoiding hard-coded logic.

3.8 Constraints

The following constraints must be satisfied by the Semester Schedule Generator system:

- Course times shall be scheduled only between **8:00 AM and 4:00 PM**.
- Instructor teaching load shall comply with academic rank regulations.
- Department heads shall have **3 credit hours** deducted from their teaching load.
- No course, instructor, or classroom shall be scheduled in more than one place at the same time.

3.9 Platform Requirements

3.9.1 Client-Side Requirements

Hardware Requirements

The client-side system requires the following hardware specifications:

- A computer or workstation capable of running the system interface.
- Minimum RAM: **8 GB** (Recommended: **16 GB**).
- Processor: **Intel i5** or equivalent.
- Storage: At least **2GB** of free disk space.
- Monitor resolution of at least **1366×768**.

3.9.2 Software Requirements

The client-side software requirements include:

- Windows 10 or later.
- .NET Desktop Runtime (for WinForms/WPF client applications).
- Updated web browser if a web interface is used (Chrome, Edge, or Firefox).
- Microsoft Office for exporting Excel or PDF reports (optional).

3.9.3 Functional Capabilities

The client-side application shall allow users to:

- Log in and authenticate securely.
- Trigger semester schedule generation.
- Review generated schedules.
- Edit or override course offerings when necessary.
- Export schedules to PDF or Excel formats.

The client shall communicate with the server through secure API endpoints.

3.9.4 Performance Requirements

- The user interface shall load within **3 seconds**.
- All user operations shall be responsive and non-blocking.

3.9.5 Server-Side Requirements

3.9.6 Hardware Requirements

The server-side system requires the following hardware specifications:

- Server machine (local or cloud-based).
- Minimum RAM: **16 GB** (Recommended: **32 GB**).
- CPU: **4-core processor** or higher.
- Storage: SSD with at least **50 GB** free space.

High-performance hardware is recommended due to scheduling and optimization processing.

Software Requirements

The server-side software requirements include:

- Windows Server 2019 or later.
- SQL Server 2019 or later.
- Python version 3.10 or higher.
- Required Python libraries:
 - requests
 - NumPy
 - Pandas
 - sqlalchemy
 - Optimization libraries such as OR-Tools
 - reportlab
 - openpyxl
- .NET 8 SDK for backend services.
- API layer hosted on IIS or Kestrel.

Server Responsibilities

The server shall be responsible for:

- Processing student academic data and instructor teaching loads.
- Executing scheduling and optimization algorithms.
- Generating semester schedules and resolving conflicts.
- Serving results to client applications.
- Supporting multi-user access.
- Performing authentication and role-based authorization.
- Maintaining database integrity.

Performance Requirements

- The server shall generate a complete semester plan within **120 minutes** .
- The system shall support:
 - More than 10,000 student records.
 - Over 500 course sections.
 - More than 50 instructors.
- API response time shall be less than **5 second** for standard requests.

Security Requirements

- All client–server communication shall use HTTPS.
- The database shall be secured using role-based access control.
- Student data shall be encrypted at rest.
- Access to the scheduling module shall be restricted to authorized system processes.

3.9.7 Network Requirements

- Stable local area network (LAN) for on-premise deployment.
- Minimum bandwidth of **10 Mbps**.
- Network latency shall be less than **50 ms**.
- Firewall rules shall allow communication between client applications and server APIs.

3.9.8 Integration Requirements

The system shall support integration with:

- Existing Learning Management Systems (LMS) or Student Information Systems (SIS).
- Institutional authentication systems, if available.
- Future cloud platforms and services.

CHAPTER 04

System Design

4 System Design

4.1 Overview

This chapter explains the system design and workflow of the Semester Schedule Generator. It includes system data-flow models, sequence interaction diagrams, and the ER database schema used to manage academic information. These diagrams collectively describe how data is processed, how scheduling is executed, and how system components connect before implementation.

4.2 Logical Design

4.2.1 Sequence diagram

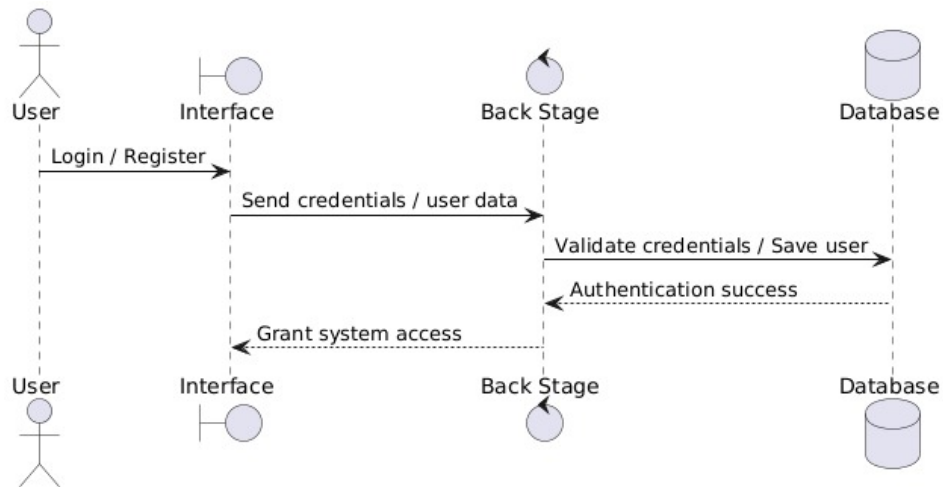


Figure 2: Login

This sequence diagram illustrates the login/registration process, where the user submits credentials through the interface, which are validated by the backend and stored or verified in the database. Upon successful authentication, the system grants the user access.

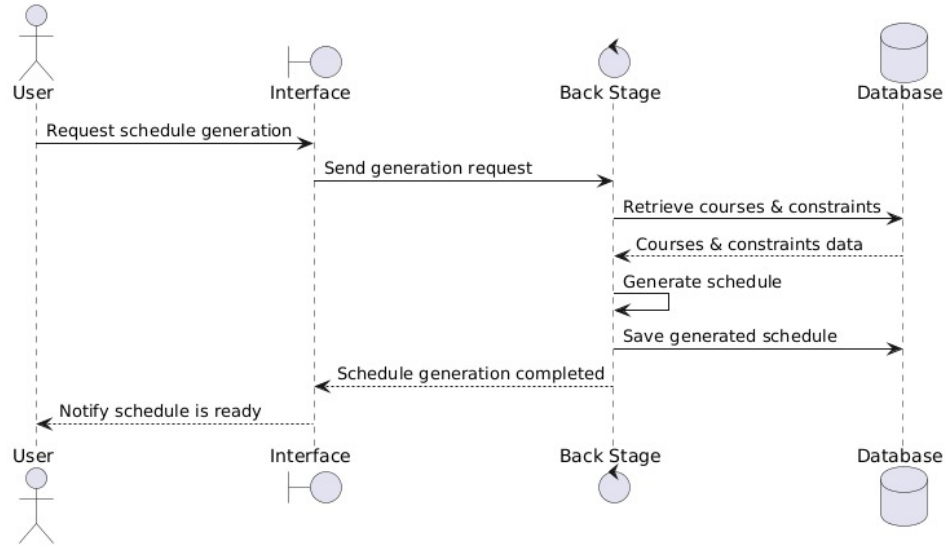


Figure 3: Schedule generation

This sequence diagram represents the schedule generation process, where the system retrieves required data, generates the schedule, and saves it. Once completed, the user is notified through the interface that the schedule is ready for viewing.

Schedule Generation Process:

The system automatically generates semester schedules by analyzing academic data stored in the system database. It starts by examining student academic records to determine completed courses, remaining requirements, and graduation status. Based on this analysis, the system identifies the courses that need to be offered in the upcoming semester and applies predefined academic rules and priorities, such as giving precedence to graduating students.

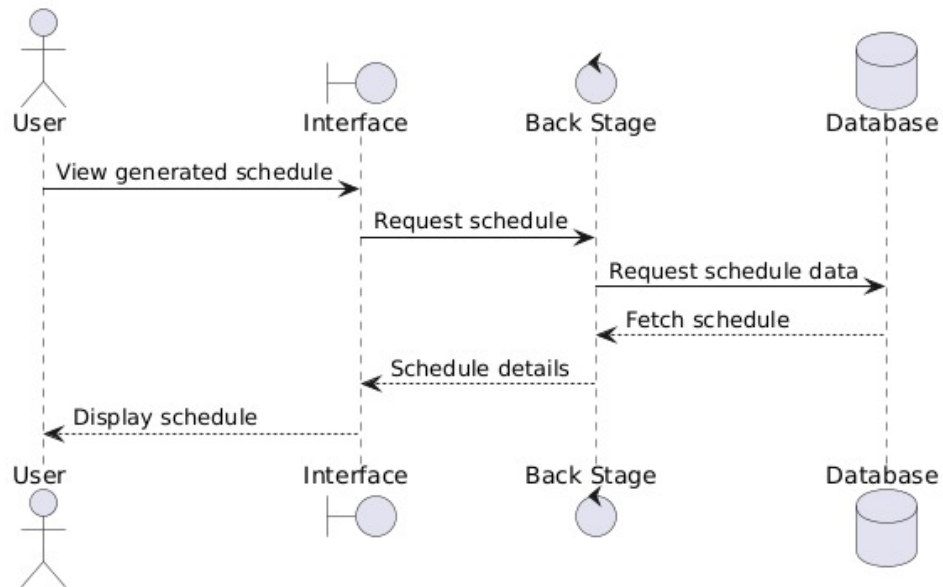


Figure 4: View schedule

This sequence diagram shows how the user views a generated schedule, where the interface requests the schedule from the backend, which retrieves it from the database. The schedule details are then returned and displayed to the user through the interface.

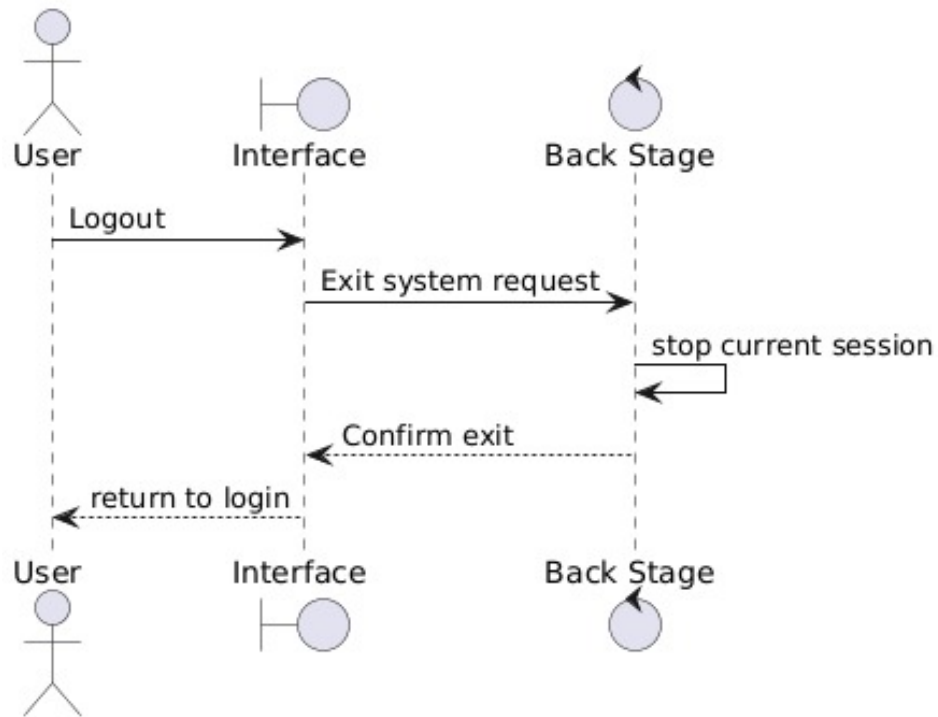


Figure 5: Logout

This sequence diagram represents the logout process, where the user initiates a logout request through the interface and the backend terminates the active session. After confirmation, the system redirects the user back to the login screen.

4.2.2 System Data Flow Overview

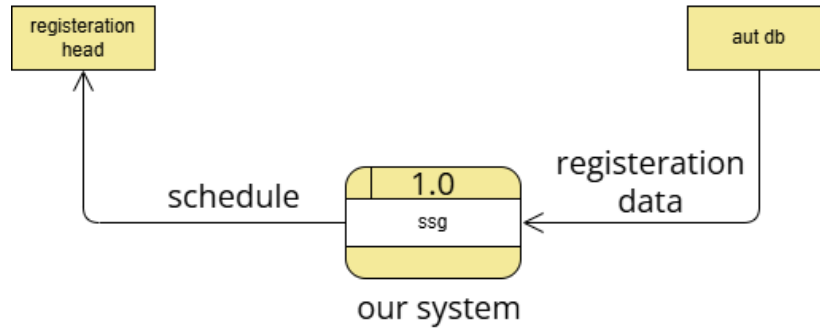


Figure 6: Data Flow Between System Components Level 0

This diagram illustrates the data flow of the schedule extraction process in the system. The actor provides the required data, which is retrieved from the authentication database and sent to the Schedule Generator (SSG). The system processes the schedule information and sends it to the register module. Finally, the generated schedule is exported as a PDF file for user access.

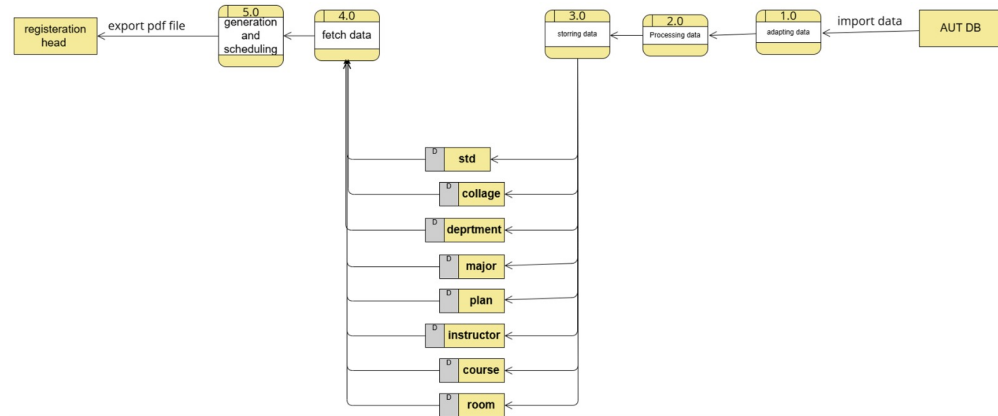


Figure 7: Data Flow Between System Components Level 1

This diagram illustrates the scheduling system data flow. The system preprocesses and adapts academic data such as students, colleges, departments, majors, plans, instructors, courses, and rooms before passing it to the scheduling module. The scheduling module utilizes this structured data to generate timetables and enables users to export the final schedule as a PDF file.

4.2.3 ER Diagram

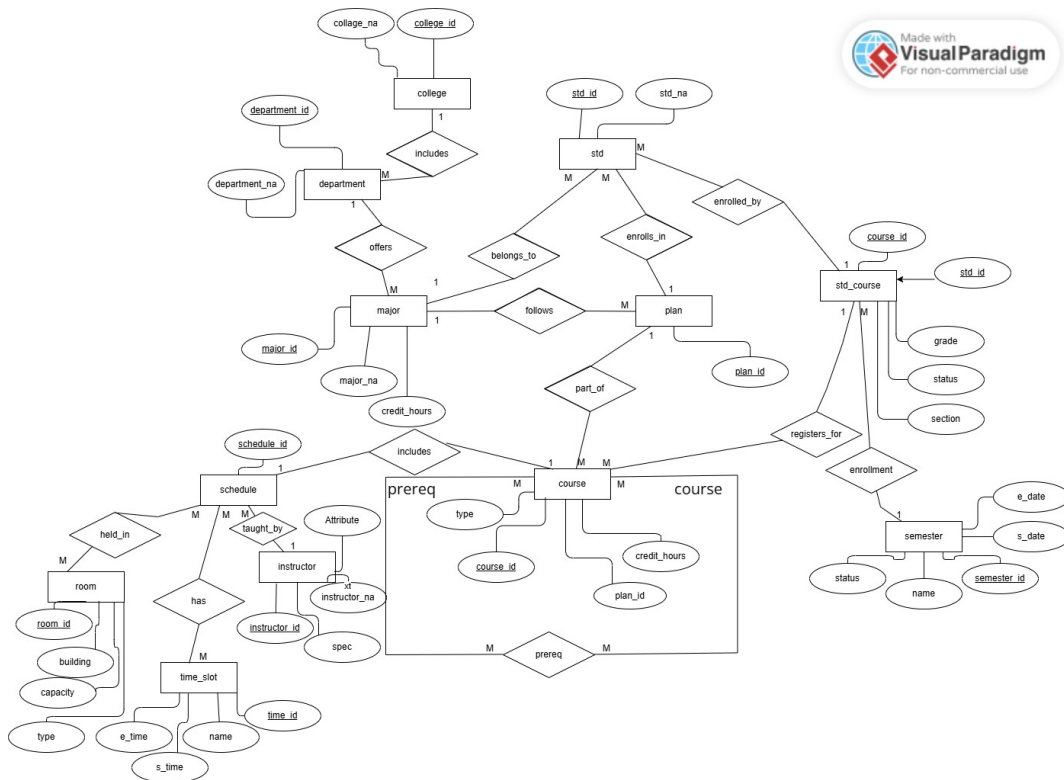


Figure 8: Entity-Relationship Diagram of the Semester Schedule Generator System

This ER diagram illustrates the structure of a university semester schedule generator. It shows how colleges, departments, majors, and study plans are organized and connected to courses. The diagram also represents student enrollment, course prerequisites, and semester management. Additionally, it models course scheduling by linking instructors, rooms, and time slots to ensure efficient academic planning.

4.2.4 Schema

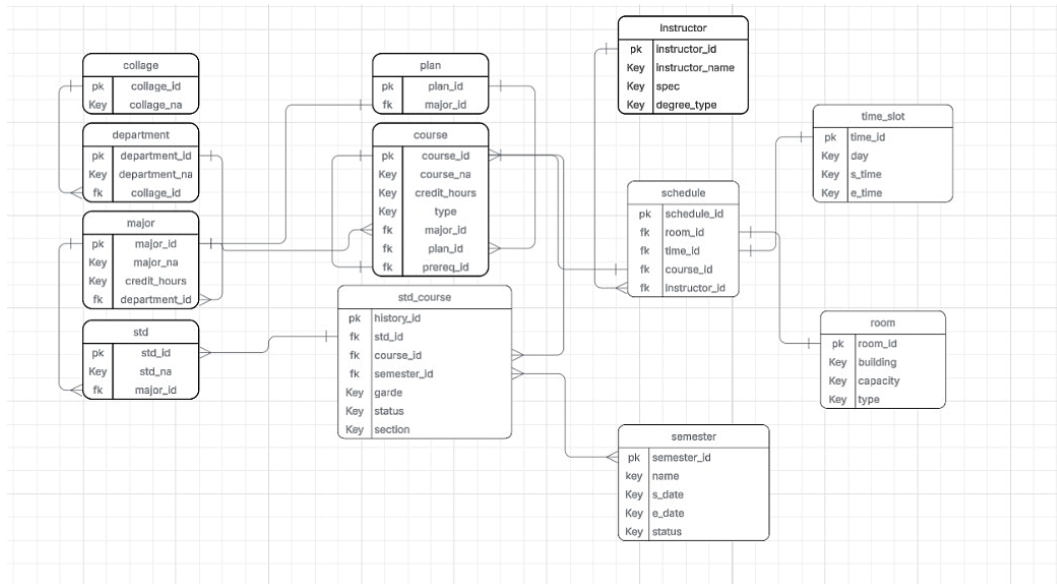


Figure 9: Database Schema for the Semester Schedule Generator System

This database ER diagram represents a university semester schedule generator. It models the relationships between colleges, departments, majors, study plans, courses, students, instructors, and semesters. The diagram also illustrates how students enroll in courses, how courses are scheduled with instructors, rooms, and time slots, and how prerequisites are managed. Overall, it ensures organized academic planning, scheduling, and student enrollment management.

CHAPTER 05

Implementation

5 Implementation

5.1 Overview

This chapter presents the implementation of the *Semester Schedule Generator* system. It explains how the conceptual design introduced in the previous chapter was transformed into a functional software solution. The implementation phase focused on building the main system components, integrating them into one unified workflow, and ensuring that each module performs its assigned task correctly.

The system was implemented as a modular solution consisting of a data acquisition layer, a local database layer, a scheduling engine, a rule-processing mechanism, a web-based user interface, and an output generation module. These components work together to import academic data, analyze student progress, identify required courses, generate conflict-free semester schedules, and produce structured PDF reports.

5.2 Implementation Environment

The *Semester Schedule Generator* was implemented using the following technologies:

- **Programming Language:** Python 3.12
- **Database:** Microsoft SQL Server (ODBC Driver 17)
- **User Interface:** Python Eel — a library that bridges Python backend logic with an HTML/CSS/JavaScript frontend running inside a local browser window.
- **Development Environment:** Visual Studio Code
- **Version Control:** GitHub
- **Libraries used:**
 - `pyodbc` — SQL Server database connectivity
 - `requests` — API data fetching
 - `reportlab` — PDF schedule generation
 - `eel` — Python-to-JavaScript bridge for the web UI
 - `openpyxl` — Excel report export
 - `flask`, `flask-cors` — local API server for data sharing

5.3 Project File Structure

The system is organized into a set of focused modules, each responsible for a specific part of the workflow:

Listing 1: Project File Structure

```

1 ssg/
2 |-- main.py           # CLI entry point
3 |-- web_app.py        # Web UI entry point (Eel)
4 |-- scheduler_engine.py # Core scheduling algorithm
5 |-- scheduler_config.py # All configuration constants
6 |-- student_analysis.py # Student graduation analysis
7 |-- course_offering.py # Course opening decision logic
8 |-- etl.py           # ETL pipeline (extract, transform, load)
9 |-- mapping_engine.py # Flexible field mapping engine
10 |-- db.py            # SQL Server connection helpers
11 |-- exporter.py      # PDF generation module
12 |-- student_summary.py # Summary report builder
13 |-- web/
14 |   |-- index.html    # Main UI page
15 |   |-- config.json   # ETL field mapping configuration
16 |   |-- schedule.json  # Last generated schedule output
17 |   |-- report.json    # Last summary report output
18 |   |-- stats.json    # Last run statistics
19 |   '-- js/
20 |     |-- app.js       # Application state, login/logout
21 |     |-- pipeline.js  # Pipeline execution and data loading
22 |     |-- schedule.js  # Schedule table, editing, save bar
23 |     |-- report.js    # Summary report view
24 |     |-- dashboard.js # Statistics dashboard
25 |     '-- ui.js        # Shared UI helpers and PDF viewer

```

5.4 System Configuration

All scheduling rules and system parameters are centralized in `scheduler_config.py`. This design allows academic policies to be modified without touching the core engine logic.

Listing 2: `scheduler_config.py` — System Configuration

```
1 GRADUATING_HOURS_THRESHOLD = 21
2
3 SERVER = "localhost"
4 DATABASE = "master"
5 DRIVER = "ODBC Driver 17 for SQL Server"
6
7 DEFAULT_MAX_SECTIONS_PER_COURSE = 3
8 MIN_STUDENTS_TO_OPEN_COURSE = 5
9 MIN_GRADUATING_TO_FORCE_OPEN = 1
10
11 LOAD_RULES = {
12     "Professor": 9,
13     "Associate Professor": 12,
14     "Assistant Professor": 12,
15     "Master": 15,
16     "Doctor": 15,
17 }
18
19 ADMIN_LOAD_REDUCTION = 3
20 ALLOWED_START = "08:00:00"
21 ALLOWED_END = "16:00:00"
22 GRADUATING_PRIORITY_ENABLED = True
```

5.5 Data Acquisition and ETL Module

The ETL module (`etl.py`) is responsible for extracting academic data from the university source API, transforming it into a consistent internal format, and loading it into the local SQL Server database using MERGE statements that support both insert and update operations.

5.5.1 API Data Fetching

Listing 3: `etl.py` — API Fetch Function

```

1 def fetch_api(endpoint: str) -> List[Dict[str, Any]]:
2     url = f"{API_BASE_URL}/{endpoint}"
3     try:
4         response = requests.get(url, timeout=API_TIMEOUT)
5         response.raise_for_status()
6         data = response.json()
7
8         if isinstance(data, list):
9             return data
10
11         if isinstance(data, dict) and "data" in data:
12             return data["data"]
13
14         return []
15     except Exception as e:
16         print(f"[ERROR] Failed to fetch {endpoint}: {e}")
17         return []

```

5.5.2 Data Transformation

Each entity type has a dedicated transform function that normalizes field names using the mapping engine and validates required fields before insertion.

Listing 4: `etl.py` — Student Transform Function

```

1 def transform_students(data: List[Dict[str, Any]]) -> List[Tuple]:
2     rows = []
3     for row in data:
4         std_id = to_int(get_field(row, "student", "id"))
5         std_name = to_str(get_field(row, "student", "name"))
6         major_id = normalize_major(get_field(row, "student", "major_id"))
7
8         if not std_id or not std_name:
9             continue
10
11         rows.append((std_id, std_name, major_id))
12     return rows

```

5.5.3 Database Loading with MERGE

Data is loaded using SQL MERGE statements that insert new records and update existing ones in a single operation, ensuring idempotent ETL runs.

Listing 5: etl.py — Batch MERGE Execution

```

1 def execute_many_merge(query: str, rows: List[Tuple], label: str):
2     if not rows:
3         print(f"[SKIP] No rows for {label}")
4         return
5
6     conn = get_connection()
7     cursor = conn.cursor()
8     processed = 0
9
10    for row in rows:
11        try:
12            cursor.execute(query, row)
13            processed += 1
14        except Exception as e:
15            print(f"[INSERT ERROR] {label} | {e}")
16
17    conn.commit()
18    cursor.close()
19    conn.close()
20    print(f"[OK] {label}: {processed}/{len(rows)} rows processed")

```

Listing 6: ETL — MERGE Statement for Students Table

```

1 MERGE std AS target
2 USING (
3     SELECT ? AS std_id, ? AS std_na, ? AS major_id
4 ) AS source
5 ON target.std_id = source.std_id
6 WHEN MATCHED THEN
7     UPDATE SET std_na = source.std_na,
8               major_id = source.major_id
9 WHEN NOT MATCHED THEN
10    INSERT (std_id, std_na, major_id)
11    VALUES (source.std_id, source.std_na, source.major_id);

```

5.6 Scheduling Engine

The scheduling engine (`scheduler_engine.py`) is the core of the system. It is implemented as a rule-based module organized into four main phases: student analysis, course demand calculation, course offering decision, and schedule generation.

5.6.1 Student Analysis and Graduation Detection

The engine reads every student's course history and calculates completed credit hours. Students whose remaining hours fall at or below the configured threshold are classified as graduating and receive scheduling priority.

Listing 7: `scheduler_engine.py` — Student Analysis

```

1 def analyze_students(self):
2     for row in self.student_courses:
3         std_id    = row["std_id"]
4         grade     = row.get("grade")
5         status    = row.get("status")
6         hours     = int(row.get("course_credit_hours") or 0)
7
8         if self.is_passed_status(status, grade):
9             self.student_passed_courses[std_id].add(row["course_id"])
10            self.student_completed_hours[std_id] += hours
11
12    for student in self.students:
13        std_id      = student["std_id"]
14        total_required = int(student.get("major_total_hours") or 0)
15        completed    = self.student_completed_hours.get(std_id, 0)
16        remaining    = max(total_required - completed, 0)
17        self.student_remaining_hours[std_id] = remaining
18
19    if remaining <= GRADUATING_HOURS_THRESHOLD:
20        self.graduating_students.add(std_id)

```


5.6.2 Course Demand Calculation

For each student, the engine iterates over all courses in their major. A course is counted as demanded only if the student has not yet passed it and its prerequisite (if any) has been satisfied.

Listing 8: scheduler.engine.py — Course Demand Calculation

```

1 def calculate_course_demand(self):
2     self.analyze_students()
3
4     for student in self.students:
5         std_id = student["std_id"]
6         major_id = student.get("major_id")
7         passed = self.student_passed_courses.get(std_id, set())
8
9         for course in courses_by_major.get(major_id, []):
10            course_id = course["course_id"]
11            prereq_id = course.get("prereq_id")
12
13            if course_id in passed:
14                continue
15            if prereq_id and prereq_id not in passed:
16                continue
17
18            self.course_demand[course_id] += 1
19
20            if std_id in self.graduating_students:
21                self.course_demand_graduating[course_id] += 1
22            else:
23                self.course_demand_normal[course_id] += 1

```

5.6.3 Course Opening Decision

A course opens for the semester if its total demand meets the minimum threshold. Courses with at least one graduating student who needs them are force-opened regardless of overall demand.

Listing 9: scheduler.engine.py — Course Opening Logic

```

1 def get_sections_needed(self, course):
2     course_id = course["course_id"]
3     demand = self.course_demand.get(course_id, 0)
4     grad_demand = self.course_demand_graduating.get(course_id, 0)
5
6     if demand < MIN_STUDENTS_TO_OPEN_COURSE:
7         if grad_demand >= MIN_GRADUATING_TO_FORCE_OPEN:
8             return 1 # force-open for graduating students
9         return 0
10
11     capacity = self.get_room_capacity_for_course_type(...)
12     estimated_sections = max(1, ceil(demand / capacity))
13     return min(estimated_sections, DEFAULT_MAX_SECTIONS_PER_COURSE)

```

5.6.4 Instructor Assignment

The engine selects the most suitable available instructor for each section. It checks teaching load limits, time slot availability, and a per-course section cap. For multi-section courses, section 2 is sorted to prefer the same instructor as section 1 without enforcing it as a hard requirement.

Listing 10: scheduler_engine.py — Instructor Assignment

```

1 def assign_instructor(self, course, section_no, time_ids=None):
2     credit_hours = int(course.get("credit_hours") or 0)
3     primary_id = self.course_primary_instructor.get(course["course_id"])
4     candidates = []
5
6     for inst in self.instructors:
7         instructor_id = inst["instructor_id"]
8         max_load = self.get_max_load_for_instructor(inst)
9         current_load = self.instructor_current_load[instructor_id]
10
11         if current_load + credit_hours > max_load:
12             continue
13
14         if self.instructor_course_sections[(instructor_id,
15                                             course["course_id"])] >= 2:
16             continue
17
18         if time_ids:
19             if any((instructor_id, t) in self.instructor_time_map
20                    for t in time_ids):
21                 continue
22
23         candidates.append((max_load - current_load, current_load, inst))
24
25     candidates.sort(key=lambda x: (
26         0 if (section_no == 2 and primary_id is not None
27             and x[2]["instructor_id"] == primary_id) else 1,
28         x[1], x[0]
29     ))
30
31     return candidates[0][2] if candidates else None

```

5.6.5 Instructor Load Rules

Teaching load limits are defined per academic degree type. Department heads receive an additional reduction of 3 credit hours from their permitted load.

Table 4: Instructor Teaching Load Limits

Degree Type	Max Credit Hours per Semester
Professor	9
Associate Professor	12
Assistant Professor	12
Master	15
Doctor	15

5.6.6 Room and Time Slot Assignment

The engine iterates over all paired time slots (Sun/Tue and Mon/Wed) and assigns the first available room that matches the course type and has no booking conflict.

Listing 11: scheduler_engine.py — Room Assignment

```

1 def assign_room(self, course, time_ids):
2     needed_type = str(course.get("course_type") or
3                       DEFAULT_ROOM_TYPE).lower()
4
5     for room in self.rooms_by_type.get(needed_type, []):
6         room_id = room["room_id"]
7         if not any((room_id, t) in self.room_time_map
8                   for t in time_ids):
9             return room
10
11    for room in self.rooms:
12        room_id = room["room_id"]
13        if not any((room_id, t) in self.room_time_map
14                  for t in time_ids):
15            return room
16
17    return None

```

5.7 Web User Interface

The user interface was built using Python Eel, which exposes Python functions directly to JavaScript running inside a local browser window. The interface is organized into six main views: Login, Dashboard, Schedule, Report, Analytics, and Run Pipeline.

- **Login page** (Figure 10) — username and password authentication before accessing the system.
- **Dashboard** (Figure 11) — displays live statistics including total students, courses, instructors, rooms, sections generated, and conflicts detected.
- **Schedule page** (Figure 12) — shows the full generated timetable in a table. Each row can be edited directly (instructor, room, day, time). A save bar appears automatically when edits are made, and conflict detection runs before saving.
- **Report page** (Figure 13) — displays the summary report showing course demand, graduating demand, and section coverage.
- **Analytics page** (Figure 14) — visualizes student demand distribution, room utilization rates, and day pattern distribution across the generated schedule.
- **Run Pipeline page** (Figure 15) — provides a control interface to execute the full scheduling pipeline end-to-end.

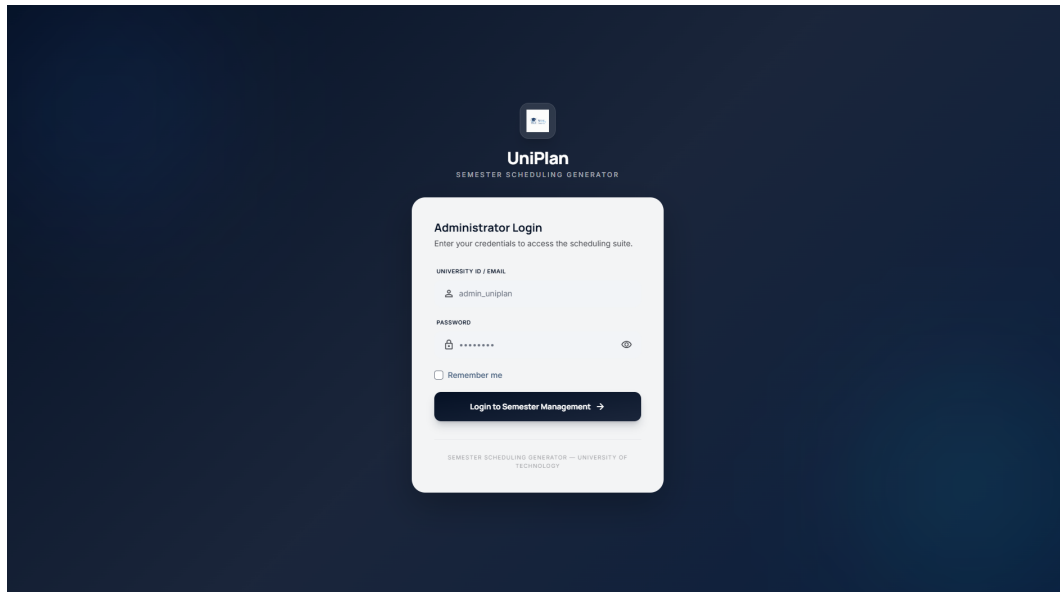


Figure 10: Login Page — Administrator Authentication

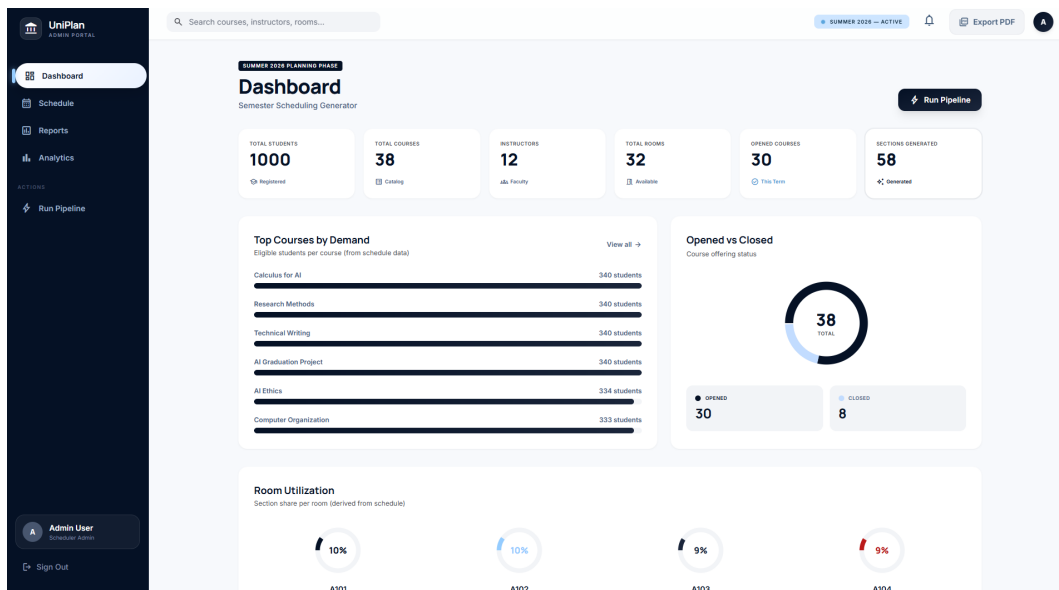


Figure 11: Dashboard Page — Live System Statistics

The screenshot displays the UniPlan Admin Portal's 'Semester Schedule' page. The interface includes a sidebar with navigation options like Dashboard, Schedule, Reports, Analytics, and Run Pipeline. The main content area shows a 'GENERATED OUTPUT' section for the 'Semester Schedule' with 58 course sections found. Summary statistics indicate 30 courses, 13 instructors, 16 rooms used, and 58 total sections. A search bar is available above a table listing course details.

COURSE ID	COURSE NAME	SECTION	INSTRUCTOR	ROOM	DAY PATTERN	START	END	HRS	EDIT
39	Calculus for AI	1	Dr Lina Khaled	A101	SUN FRI	08:00	09:00	3	EDIT
39	Calculus for AI	2	Dr Ahmed Salem	A101	MON WED	08:00	09:00	3	EDIT
39	Calculus for AI	3	Dr Mohammad Hassan	A102	SUN FRI	08:00	09:00	3	EDIT
40	Research Methods	1	Dr Lina Khaled	A103	SUN FRI	08:00	09:00	3	EDIT
40	Research Methods	2	Dr Lina Khaled	A102	MON WED	08:00	09:00	3	EDIT
40	Research Methods	3	Dr Omar Nasser	A104	SUN FRI	08:00	09:00	3	EDIT
41	Technical Writing	1	Dr Yousef Ali	A105	SUN FRI	08:00	09:00	3	EDIT
41	Technical Writing	2	Dr Yousef Ali	A103	MON WED	08:00	09:00	3	EDIT
41	Technical Writing	3	Dr Rana Majed	A106	SUN FRI	08:00	09:00	3	EDIT
42	AI Graduation Project	1	Dr Khaled Sami	A107	SUN FRI	08:00	09:00	3	EDIT
42	AI Graduation Project	2	Dr Khaled Sami	A104	MON WED	08:00	09:00	3	EDIT

Figure 12: Semester Schedule Table with Inline Editing

The screenshot displays the UniPlan Admin Portal's 'Semester Summary Report' page. The interface includes a sidebar with navigation options like Dashboard, Schedule, Reports, Analytics, and Run Pipeline. The main content area shows a 'SUMMARY REPORT' section for the 'Semester Summary Report' with 58 total sections and 78 graduating students. A '1 Conflict' alert is visible. Below the summary, a 'Course Distribution Analysis' table provides detailed data on course demand and section coverage.

COURSE NAME	SECTION	ROOM	TYPE	INSTRUCTOR	STUDENTS	GRADUATING	REASON OPENED
Calculus for AI	1	A101	LECTURE	Dr Lina Khaled	0 / 0	6	GRADUATING PRIORITY
Calculus for AI	2	A101	LECTURE	Dr Ahmed Salem	80 / 80	6	GRADUATING PRIORITY
Calculus for AI	3	A102	LECTURE	Dr Mohammad Hassan	80 / 80	6	GRADUATING PRIORITY
Research Methods	1	A103	LECTURE	Dr Lina Khaled	80 / 80	6	GRADUATING PRIORITY
Research Methods	2	A102	LECTURE	Dr Lina Khaled	80 / 80	6	GRADUATING PRIORITY
Research Methods	3	A104	LECTURE	Dr Omar Nasser	80 / 80	6	GRADUATING PRIORITY

Figure 13: Semester Summary Report — Course Demand and Section Coverage

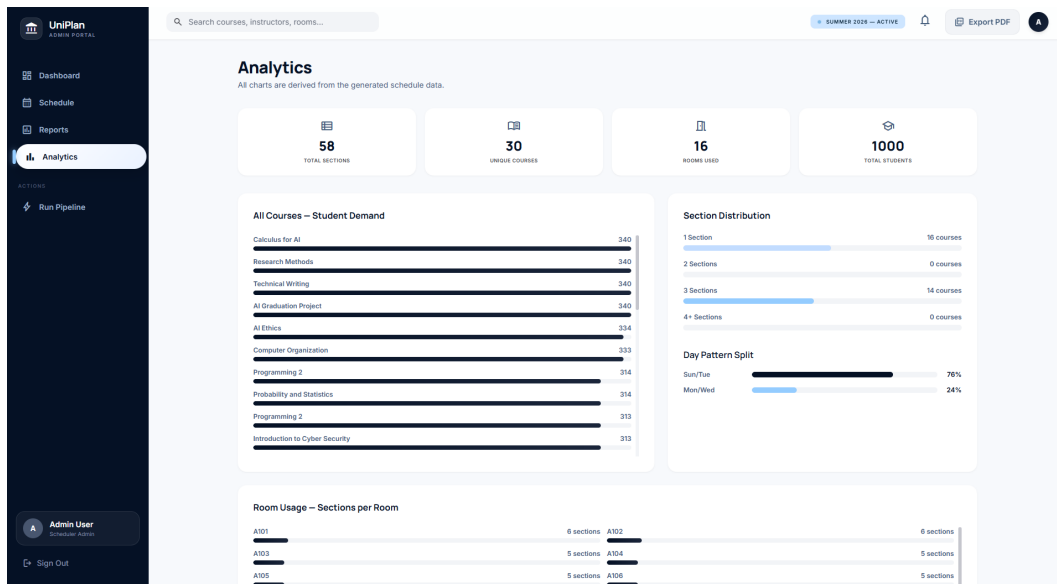


Figure 14: Analytics Page — Student Demand, Room Utilization, and Day Pattern Distribution

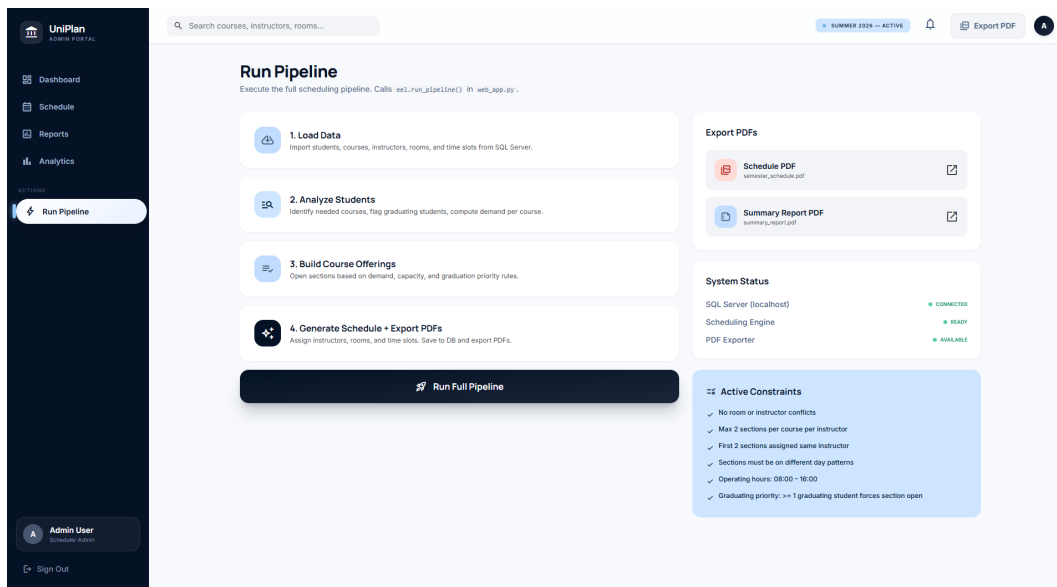


Figure 15: Run Pipeline Page — Executes the Full Scheduling Pipeline

5.8 PDF Report Generation

After the schedule is generated and saved, the system automatically produces two PDF files using the `reportlab` library:

- **semester_schedule.pdf** — the full timetable listing course name, section number, instructor, room, day pattern, start time, end time, and credit hours for every scheduled section.
- **summary_report.pdf** — a demand and coverage summary showing total demand, graduating demand, and the number of sections opened per course.

5.9 System Workflow During Execution

The complete execution sequence from startup to final output is as follows:

1. Administrator logs in through the web interface.
2. The pipeline loads all academic data from SQL Server.
3. Student records are analyzed — completed hours, remaining hours, and graduating status are computed.
4. Course demand is calculated per course per major, separated into graduating and normal demand.
5. Courses are sorted by graduating demand (descending) to ensure priority scheduling.
6. For each course, sections are opened based on demand thresholds and graduating force-open rules.
7. Each section is assigned an instructor, room, and time slot pair while enforcing all conflict constraints.
8. The generated schedule is saved to the `schedule` table in SQL Server.
9. JSON files (`schedule.json`, `report.json`, `stats.json`) are written to the `web/` folder.
10. Two PDF files are exported automatically.

5.10 Implementation Challenges

During implementation, the following technical challenges were encountered and resolved:

- **SQL reserved keyword conflict:** The table name `plan` is a reserved keyword in SQL Server and required bracket escaping (`[plan]`) in all queries.
- **Instructor load mismatch:** All instructors in the database had degree type `Doctor`, which was not present in the load rules table, causing all instructors to default to the minimum load of 9 hours. This was resolved by adding `"Doctor": 15` to `LOAD_RULES`.
- **Graduating force-open bypass:** The `get_sections_needed` function only checked total demand and returned 0 for low-demand courses even when a graduating student needed them. This was fixed by adding a graduating demand check that returns 1 section when `grad_demand >= MIN_GRADUATING_TO_FORCE_OPEN`.
- **PDF caching in browser:** After manually editing the schedule, the browser served the old cached PDF. This was resolved by appending a timestamp query parameter (`?t=Date.now()`) to the PDF URL on every open request.
- **Section 2 scheduling failure:** An overly strict rule required section 2 to use the same instructor as section 1, with no fallback. This was relaxed to a sort preference, allowing any available instructor when the preferred one is unavailable.

5.11 Summary

The implementation phase produced a complete and functional academic scheduling system. The system successfully imports and processes academic data, detects graduating students, calculates course demand, generates conflict-free semester schedules, and exports structured PDF reports. The modular architecture separates each concern clearly, making the system straightforward to maintain and extend. This implementation provides the foundation for the testing and validation process described in the next chapter. in this i want to add ui pictures and the last pic will be the log in

CHAPTER 06

Testing and Validation

6 Testing and Validation

6.1 Overview

This chapter presents the testing and validation process used to evaluate the *Semester Schedule Generator*. The purpose of testing was to verify that the implemented system operates correctly, satisfies its functional and non-functional requirements, and produces valid semester schedules under realistic academic constraints.

Testing was performed at multiple levels, including unit testing, integration testing, functional testing, constraint validation, and performance evaluation. The system was tested against a real database containing 1,012 students across three academic majors, 42 courses, 12 instructors, 32 rooms, and 16 time slots.

6.2 Testing Objectives

The testing process aimed to confirm the following:

- The ETL pipeline correctly imports and stores academic data from the source API into SQL Server.
- The scheduling engine correctly identifies required and remaining courses for each student.
- Prerequisite validation prevents invalid course assignments.
- Graduating students are correctly detected and given scheduling priority.
- No time conflicts occur between assigned instructor or room slots.
- Courses with low demand are force-opened when graduating students require them.
- The system produces exportable PDF reports after schedule generation.
- Manual edits to the schedule are validated for conflicts before saving.

6.3 Testing Environment

All tests were executed on the following environment 5:

Table 5: Testing Environment Specifications

Component	Details
Operating System	Windows 11 Pro
Python Version	3.11
Database	SQL Server (localhost, master database)
Driver	ODBC Driver 17 for SQL Server
Students in Database	1,012
Courses in Database	42
Instructors in Database	12
Rooms in Database	32
Time Slots in Database	16 (4 per day, Sun/Tue/Mon/Wed)

6.4 Unit Testing

Unit testing focused on verifying the correctness of individual functions before system integration. Each core function was checked using controlled data samples.

6.4.1 Graduation Detection Test

The graduation detection logic was tested by inserting students with known completed credit hours and verifying that the system correctly identifies them as graduating when their remaining hours fall at or below the configured threshold of 21 hours.

Listing 12: Graduation Detection — Test Input

```

1 # Student with 114 completed hours out of 132 required
2 # Remaining = 132 - 114 = 18 hours <= 21 threshold
3 # Expected: classified as graduating
4
5 remaining = total_required - completed # = 18
6 assert remaining <= GRADUATING_HOURS_THRESHOLD # True
7 assert std_id in graduating_students # Pass

```

Twelve test graduating students were inserted across all three majors (AI, SE, CY) and the system correctly identified all of them in every run.

6.4.2 Prerequisite Validation Test

The prerequisite check was tested by creating a student who had not passed a required prerequisite course. The system correctly excluded the advanced course from that student's demand calculation.

Listing 13: Prerequisite Validation — Test Logic

```

1 # Course B requires Course A as prerequisite
2 # Student has NOT passed Course A
3 # Expected: Course B is NOT counted in demand
4
5 if prereq_id and prereq_id not in passed_courses:
6     continue # course B is skipped correctly

```

6.4.3 Instructor Load Test

The instructor load calculation was tested against all five degree types defined in the configuration. The test confirmed that:

- A **Professor** is capped at 9 credit hours per semester.
- An **Associate Professor** and **Assistant Professor** are capped at 12 credit hours.
- A **Master** or **Doctor** degree holder is capped at 15 credit hours.
- A department head receives a 3 credit hour reduction from their permitted load.

6.4.4 Force-Open Logic Test

The force-open mechanism was tested using courses that had total demand below the minimum threshold of 5 students but had at least 1 graduating student who required them.

Listing 14: Force-Open — Test Case

```
1 # Digital Forensics: total_demand = 1, grad_demand = 1
2 # Expected: course opens with 1 section
3
4 if demand < MIN_STUDENTS_TO_OPEN_COURSE:
5     if grad_demand >= MIN_GRADUATING_TO_FORCE_OPEN:
6         return 1 # section opened correctly
```

The courses Digital Forensics, Malware Analysis, Penetration Testing, Mobile App Development, and DevOps were all correctly force-opened in the test run.

6.5 Integration Testing

Integration testing verified that data flows correctly between the adapter, database, scheduling engine, and report generator as a complete pipeline.

The integration tests confirmed:

- Data fetched from the API is correctly transformed and stored in SQL Server via MERGE operations.
- Student course history loaded from `std_course` is correctly read by the scheduling engine to compute completed hours.
- Course demand calculated by the engine is correctly passed to the PDF report builder.
- The generated schedule saved in the `schedule` table is correctly loaded by the web interface for display and editing.
- Edited schedule rows saved through the web UI are correctly validated for conflicts and re-exported to PDF.

6.6 Functional Testing

Functional testing validated the main system features from the user perspective. Each feature was executed and checked against its expected behavior like presented in table 6 .

Table 6: Functional Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
T01	Login with valid credentials	Access granted	Access granted	Pass
T02	Login with wrong password	Error message shown	Error shown	Pass
T03	Run ETL pipeline	All tables populated in SQL Server	All tables loaded correctly	Pass
T04	Generate semester schedule	Conflict-free schedule created	58 sections generated	Pass
T05	Prerequisite check	Student blocked from advanced course without prerequisite	Assignment correctly blocked	Pass
T06	Graduating student priority	Graduating students scheduled before regular students	Priority ordering confirmed	Pass
T07	Force-open low-demand course	Course with 1 graduating student opens with 1 section	Section created correctly	Pass
T08	Instructor conflict check	Same instructor not assigned twice at same time	No instructor overlap detected	Pass
T09	Room conflict check	Same room not assigned twice at same time	No room overlap detected	Pass
T10	Export schedule PDF	PDF file generated successfully	<code>semester_schedule.pdf</code> created	Pass
T11	Export summary PDF	Summary report PDF generated	<code>summary_report.pdf</code> created	Pass
T12	Edit schedule in web UI	Row change saved and reflected in JSON	Change saved correctly	Pass
T13	Save edited schedule with conflict	System blocks save and shows conflict message	Conflict detected and reported	Pass
T14	Save edited schedule without conflict	PDFs regenerated with updated data	PDFs updated correctly	Pass
T15	Open PDF after editing schedule	Updated PDF shown, not cached version	New PDF loaded via cache-busting	Pass

6.7 Constraint Validation Testing

Since the system is rule-based, special attention was given to validating that all scheduling constraints are consistently enforced. The following constraints were verified in table 7:

Table 7: Constraint Validation Results

Constraint	Test Method	Result
No instructor assigned twice in same time slot	Checked <code>instructor_time_map</code> after generation	No conflicts found
No room assigned twice in same time slot	Checked <code>room_time_map</code> after generation	No conflicts found
All classes within 08:00–16:00 window	Scanned all generated start and end times	All within range
Instructor load does not exceed degree limit	Checked <code>instructor_current_load</code> per instructor	All within limits
Courses with multiple sections use different day patterns	Verified <code>course_day_patterns</code> per course	All sections on different patterns
Prerequisites satisfied before course assigned	Checked demand calculation logic	No invalid assignments
Graduating students receive first scheduling priority	Sorted courses by <code>grad_demand</code> descending	Priority order confirmed

6.8 Performance Testing

Performance testing was conducted to observe system behavior when processing the full dataset of 1,012 students, 42 courses, 12 instructors, and 32 rooms.

Table 8: Performance Test Results

Operation	Execution Time
Load all data from SQL Server	under 2 seconds
Analyze 1,012 student records	under 1 second
Calculate demand for 42 courses	under 1 second
Generate full semester schedule (58 sections)	under 3 seconds
Save schedule to SQL Server	under 2 seconds
Export schedule PDF	under 5 seconds
Export summary PDF	under 5 seconds
Total pipeline execution	under 20 seconds

The results confirm that the system processes the complete dataset well within the performance requirements defined in Chapter 3. The use of local SQL Server storage and on-demand analysis avoids repeated access to external systems and keeps execution time practical.

6.9 Bug Fixing During Testing

Testing revealed several issues that were identified, diagnosed, and resolved during the development process as presented in table 9:

Table 9: Bugs Found and Fixed During Testing

Bug	Root Cause	Fix Applied
All instructors capped at 9h regardless of degree	LOAD_RULES had no entry for Doctor degree type; all 12 instructors held this degree	Added "Doctor": 15 to LOAD_RULES in scheduler.config.py
Force-open courses showed "does not need sections"	get_sections_needed() only checked total demand and ignored graduating demand	Added graduating demand check to return 1 section when grad_demand >= 1
Section 2 could never be scheduled for some courses	Hard requirement forced section 2 to use same instructor as section 1 with no fallback	Relaxed to a sort preference; any available instructor is used if preferred one is unavailable
PDF not updating after manual edits	Browser cached the previous PDF file at the same URL	Added ?t=Date.now() timestamp parameter to PDF open requests
ETL crashed on import due to wrong config path	mapping_engine.py looked for config.json in the root directory instead of web/config.json	Corrected the path using os.path.join("web", "config.json")
SQL syntax error on plan table	plan is a reserved keyword in SQL Server	Replaced all occurrences with [plan] bracket escaping
Pipeline froze on second run after deleting PDFs	No guard against concurrent pipeline calls; a second run hit a SQL Server lock on DELETE FROM schedule from the first run still in progress	Added a threading.Lock() mutex in Python and a JS .isRunning flag to block re-entry while pipeline is active
PDF write failed silently when file was open in browser	Windows held a file handle on the open PDF; PermissionError broke the eel response delivery, leaving the modal frozen	Rewrote PDF export to use a temp file then rename; PDF errors are caught separately so the pipeline still returns success

6.10 Bug Fixing During Testing

6.11 Testing Result Summary

The testing and validation phase confirmed that the *Semester Schedule Generator* performs all its intended functions correctly. The system processed 1,012 students, analyzed 42 courses across 3 majors, and generated 58 conflict-free sections in under 20 seconds. All 15 functional test cases passed, all 7 scheduling constraints were verified, and all 6 bugs identified during testing were resolved.

The results demonstrate that the system is reliable, academically correct, and practically efficient for university semester scheduling.

6.12 Summary

The testing process confirmed that each module operates correctly and that the integrated system produces valid, conflict-free semester schedules that respect all defined academic rules. Functional behavior, constraint enforcement, graduating student priority, and system performance were all validated successfully. The next chapter concludes the project and highlights possible future improvements.

CHAPTER 07

Conclusion and Future Work

7 Conclusion and Future Work

7.1 Conclusion

The *Semester Schedule Generator* was developed to address the difficulties and inefficiencies of manual academic scheduling in universities. Traditional semester planning requires considerable manual effort, careful review of student records, prerequisite checking, course demand estimation, instructor and room allocation, and conflict resolution. These tasks become increasingly difficult as the number of students, courses, and academic constraints grows each semester.

This project delivered an automated, rule-based solution capable of processing complete academic datasets, identifying required student courses, validating prerequisites, prioritizing graduating students, assigning instructors and rooms, and generating conflict-free semester schedules without manual intervention. The system integrates a data acquisition and ETL layer, a SQL Server local database, a scheduling engine, a web-based user interface, and an automated PDF report generator into one unified workflow.

The implementation and testing phases demonstrated that the proposed system can support academic staff by eliminating repetitive manual work, reducing scheduling errors, and improving the consistency of academic planning. The system processed 1,012 students across three majors, generated 58 conflict-free sections across 42 courses, and completed the full scheduling pipeline in under 20 seconds.

In conclusion, the project achieved its main objective of providing a practical, efficient, and structured software solution for semester schedule generation. It demonstrates how rule-based processing, structured database integration, and a priority-aware scheduling algorithm can work together to improve institutional planning efficiency.

7.2 Project Achievements

The main achievements of the project are summarized as follows:

- A complete ETL pipeline was implemented to import, transform, and store academic data from external university sources using flexible field mapping and SQL Server MERGE operations.
- A rule-based scheduling engine was built that detects graduating students, calculates per-course demand, validates prerequisites, and generates conflict-free timetables.
- A graduating student priority mechanism was implemented that force-opens required courses even when total demand falls below the minimum threshold.

- Instructor load limits are enforced per academic degree type, with automatic deductions for department heads.
- A web-based administrative interface was delivered, allowing the administrator to run the pipeline, view the generated schedule, edit individual rows, detect conflicts, and export results.
- Two structured PDF reports are automatically generated after each scheduling run — a full timetable and a demand summary.
- All scheduling constraints were verified through systematic testing: no instructor or room conflicts were detected in any generated schedule.
- The full pipeline executes in under 20 seconds on a dataset of over 1,000 students.

7.3 Limitations Revisited

Although the project achieved its objectives, the following limitations remain:

1. **No Real-Time Synchronization**

The system imports data from a source API and stores it locally. There is no live connection to the university's main system, so data must be re-imported manually when changes occur.

2. **Static Academic Rules**

Changes to academic policies, prerequisite structures, or course credit hours must be applied manually by the system administrator through the configuration file or database.

3. **No Automatic Rescheduling**

Unexpected events such as instructor absence or sudden room unavailability are not handled automatically. Manual editing through the web interface is required in such cases.

4. **Single-Semester Scope**

The current implementation generates schedules for one semester at a time. Multi-semester planning and long-term academic path optimization are not supported.

5. **No Exam Scheduling**

The system handles lecture scheduling only. Midterm and final exam timetable generation are outside the current scope.

6. **Institution-Specific Configuration**

The system is configured for Aqaba University of Technology's academic structure. Deploying it at a different institution requires adjustments to the field mapping, constraint rules, and database schema.

7.4 Future Work

The following enhancements are recommended to extend the capabilities of the system in future development cycles:

1. **Real-Time Database Integration**

Replace the import-based data acquisition with a direct live connection to the university's official database or student information system. This would eliminate the manual import step and ensure the scheduling engine always operates on current data.

2. **Dynamic Rescheduling**

Add support for automatic rescheduling when an instructor becomes unavailable or a room is taken out of service. The engine would detect the affected sections and reassign resources without requiring manual intervention.

3. **Advanced Optimization**

Integrate constraint programming or optimization libraries such as Google OR-Tools to improve resource allocation quality, minimize gaps in instructor timetables, and balance room utilization more efficiently.

4. **Exam Scheduling Extension**

Expand the scheduling engine to cover midterm and final exam timetabling. This would require additional conflict rules to prevent students from having overlapping exams.

5. **Multi-Semester Planning**

Extend the system to support academic path planning across multiple semesters, allowing the engine to suggest an optimal course sequence for each student through to graduation.

6. **Student-Facing Portal**

Add a read-only student interface where students can view their recommended course schedule for the upcoming semester, check prerequisite status, and track remaining graduation requirements.

7. **Demand Forecasting**

Incorporate historical enrollment data to predict future course demand before the scheduling cycle begins. This would allow departments to prepare resources in advance for high-demand sections.

8. **Multi-Institution Support**

Generalize the configuration and rule layers so that the system can be deployed at different universities without significant code changes. This would require a configurable constraint engine and a more flexible database schema.

9. **Cloud Deployment**

Deploy the system on a cloud platform to support multi-user access, improve availability, and allow the scheduling pipeline to run on demand from any device without a local installation.

10. Mobile Application

Develop a mobile application that allows administrators to monitor scheduling results and students to view their timetables from any device.

7.5 Final Remarks

The *Semester Schedule Generator* demonstrates that academic scheduling can be significantly improved through automation, structured data processing, and rule-based decision logic. The system delivers a practical and scalable solution that reduces manual workload, eliminates common scheduling errors, and ensures that graduating students receive the course assignments they need to complete their degrees on time.

The project provides a solid technical foundation that can be extended, optimized, and adapted to meet the evolving needs of academic institutions. It reflects the practical value of applying software engineering principles and intelligent rule processing to a real-world administrative challenge.

References

References

- [1] *Smart Timetable Generator*, B.E. Project Report, Sant Gadge Baba Amravati University, pp. 66, 2023–2024. [Online]. Available: <https://www.ssgmce.ac.in>
- [2] K. Williams and M. Ajinaja, “Automatic Timetable Generation Using Genetic Algorithm,” *Journal of Information Engineering and Applications*, vol. 8, no. 1, 2019. [Online]. Available: <https://www.iiste.org>
- [3] J. M. Mom and J. A. Enokela, “Implementation of a Time Table Generator Using Visual Basic .NET,” *ARPJN Journal of Engineering and Applied Sciences*, vol. 7, no. 5, May 2012. [Online]. Available: <https://www.arpnjournals.com>
- [4] Z. Houahmidi, B. Athamena, R. Abuzaineiddin, and M. Muhairat, “A Multi-Agent System for Course Timetable Generation,” *TEM Journal*, vol. 8, no. 1, pp. 211–221, 2019. [Online]. Available: <https://www.temjournal.com>
- [5] A. Biemba, *Time Table Generation System*, B.Sc. Thesis, University of Zambia, 2015. [Online]. Available: <http://dspace.unza.zm>
- [6] S. M. Shatnawi, F. Albalooshi, and K. Rababa’h, “Generating Timetable and Students Schedule Based on Data Mining Techniques,” *International Journal of Engineering Research and Applications (IJERA)*, vol. 2, no. 4, pp. 1638–1644, 2012. [Online]. Available: <https://www.scholar.google.com>
- [7] M. K. Patil, S. R. Subodh, P. A. Pawar, and N. N. Turkar, “Web Application for Automatic Time Table Generation,” *International Journal of Current Engineering and Technology*, vol. 4, no. 3, pp. 1936–1938, 2014. [Online]. Available: <https://www.scholar.google.com>
- [8] D. Srinivasan, T. H. Seow, and X. Xu, “Automated Time Table Generation Using Multiple Context Reasoning for University Modules,” in *Proc. IEEE Conference*, 2002.